AI110 | Foundations of AI Engineering

Foundations of AI Engineering Spring 2026 (@ Section 1B | Wednesdays 6PM - 8PM PT)
Personal Member ID#: **67716**

Welcome

Getting Started

▶ Course Info

AI Prompting Guide

Course Progress

Week 1

Week 2

Week 3

Week 4

Week 5

Week 6

Week 7

Week 8

Week 9

Week 10

▶ Project Grading

Overview Tinker Show

📬 Reminder: Project 1 is due by **Monday, March 16th at 2:59AM EDT.**

Show What You Know: Game Glitch Investigator

ℹ️ Project Overview

⏰ **~3 hours**

Your AI pair programmer decided to be helpful and built you a Python game. Only problem? It's full of glitches. Sometimes the game never ends, the hints lie, or the score goes haywire. Your mission: become the **Game Glitch Investigator** -- a developer who can diagnose, explain, and repair AI-generated code responsibly. You'll use VS Code Copilot to find bugs, plan refactors across multiple files, and generate automated tests.

🎯 Goals

By completing this project, you will be able to...

- [ ] Identify syntax, logic, and runtime bugs in AI-generated Python code.
- [ ] Use AI tools critically to propose, test, and verify debugging solutions.
- [ ] Apply readable, testable fixes while using AI to generate `pytest` cases for verification.
- [ ] Reflect on human judgment, identifying when to accept, modify, or reject AI-generated plans and code.

📝 Project Instructions

▾ Phase 1: Glitch Hunt

⏰ **~45 mins**

In this phase, you'll fork and set up your project, run the buggy game, and investigate what's going wrong. Your goal is to observe, record, and reason — not to fix yet — so you can understand how the game behaves and where the AI's logic starts to break down.

Setup: Get Your Starter Code

- [ ] Go to the **Game Glitch Investigator** repo.
- [ ] Click **Fork** to create your own copy under your GitHub account, then clone your fork to your computer.
- [ ] Clone the fork to your computer, then open the cloned folder in VS Code.

Step 1: Run and Observe

- [ ] Find the file named `app.py`.
- [ ] Install the required libraries by typing `pip install -r requirements.txt` in your terminal.
- [ ] Run the game by typing `python -m streamlit run app.py` in your terminal.
- [ ] Play the game at least twice using different inputs. Watch what happens carefully! Note any crashes, strange outputs (like hints that don't match your guess), or disappearing scores.

> Think you're running into bugs before you should? Use this Getting Started video as a sanity check that what you're seeing is intended!

Step 2: Document the Glitches

- [ ] Identify at least three bugs or issues you noticed while playing.
- [ ] Open the `reflection.md` file.
- [ ] Use the section **1. What was broken when you started?** to document these bugs. For each, describe what you expected to happen versus what actually happened.

Step 3: Ask AI for Help

- [ ] Open Copilot in VS Code.
- [ ] Use the `#codebase` or `#file` variables (e.g., `#file:app.py` and `#file:logic_utils.py`) to give the AI context of your project code.

> If you aren't sure why a specific line is there, highlight it and use **Inline Chat** (Cmd+I / Ctrl+I) to ask: "Explain this logic step-by-step."

- [ ] Describe one specific glitch you found and ask the AI to explain the underlying logic causing it.

📍 **Checkpoint**: You've forked the repo, successfully run the app via Streamlit, identified three clear issues in `reflection.md`, and used Copilot's workspace awareness to understand the buggy logic.

Close Section

▾ Phase 2: Investigate and Repair

⏰ **~60 mins**

Now that you know what's broken, it's time to start fixing it -- thoughtfully. You'll use AI tools to help you test ideas, repair code across files, and verify that your game finally works as intended. The focus is on slow, careful debugging and documenting how you and the AI each contributed to the solution.

Step 1: Target and Plan Your Fixes

- [ ] Look back at the three bugs you identified in Phase 1. Choose **two** to fix first.
- [ ] In your code (`app.py` or `logic_utils.py`), add a comment like `# FIXME: Logic breaks here` where you believe the issue is located.

  Marking the "crime scene" in your code gives you a specific point to reference when using **Inline Chat**.

- [ ] For each bug, start a **New Chat Session** in the Copilot Chat view to keep the AI focused on one problem at a time.
- [ ] Use context variables like `#file:app.py` , `#file:logic_utils.py` , or `#codebase` to ensure Copilot sees the relationship between your UI and logic files.

Step 2: Refactor and Fix

- [ ] The starter code often mixes game logic with UI code in `app.py` . Use **Agent Mode** to refactor core logic (like `check_guess` or `parse_guess`) into `logic_utils.py` .

  When using Agent mode, give a multi-step instruction like: "Move the `check_guess` function to `logic_utils.py` , update the logic to fix the high/low bug, and update the import in `app.py` .

- [ ] Since Agent mode applies edits directly, you must carefully review the **diff** (the comparison of old vs new code) for every file it touched.

  If the Agent's changes look wrong or too complex, don't be afraid to click "Undo" or "Discard" and try again with a more specific prompt.

- [ ] If the Agent's fix is 90% correct but has a small error, highlight that specific line and use **Inline Chat** to ask for a targeted correction.

Step 3: Test-Driven Verification

- [ ] Ask Copilot to generate a `pytest` case in `test/test_game_logic.py` that specifically targets the bug you just fixed.
- [ ] Open your terminal and run `pytest`. Confirm that your new test passes along with the existing starter tests.

  A good test is simple. For example, if you fixed a "Too High" hint, your test should verify that a guess of `60` against a secret of `50` returns the "Too High" outcome.

- [ ] Run the app with `streamlit run app.py` to confirm the fix works in the live game.

Step 4: Document Your Work

- [ ] Near each fix in your code, add a short comment explaining how you and the AI collaborated (e.g., `#FIX: Refactored logic into logic_utils.py using Copilot Agent mode`).
- [ ] Open `reflection.md`.
- [ ] Use the section **2. How did you use AI as a teammate?** to document two types (correct and incorrect/misleading) of AI suggestions you received. For each, clearly explain:
  - [ ] What the AI suggested.
  - [ ] Whether the suggestion was correct or incorrect/misleading.
  - [ ] How you verified the result in the code or game.
- [ ] Use the section **3. Debugging and testing your fixes** to record how you verified your repairs.
- [ ] Use Copilot's **Generate Commit Message** feature in the Source Control tab to create a clear summary of your repair.
- [ ] Push your changes: `git push origin main`.

📍 **Checkpoint**: Your game now runs smoothly! You have moved core logic into `logic_utils.py`, added at least one new automated test, and documented your AI collaboration process directly in the code.

Close Section

Phase 3: Reflection and README

⏰ **~30 mins**

In this final phase, you'll pull everything together -- your fixed game, your debugging notes, and your insights about working with AI. You'll finalize your documentation to explain what you did and capture what you learned about both the code and the AI-assisted debugging process.

Step 1: Finalize your README

- [ ] Open `README.md`.
- [ ] Update the **"Demo"** section.
- [ ] Fill in the **"Document Your Experience"** section.
- [ ] If you completed **Challenge 1: Advanced Edge-Case Testing**, include a screenshot of your `pytest` results in the README showing the tests passing.

💡 **Tip:** You can take a screenshot using your computer's built-in snipping tool or screen capture shortcut.

Step 2: Write Your Reflection

- [ ] Open `reflection.md`.
- [ ] Complete the final reflection questions in the file.

> Be honest! If the AI gave you a bad suggestion that you had to reject, explain why. This shows you were in control of the process, not just following the AI blindly.

Step 4: Final Commit and Push

- [ ] Use Copilot's **Generate Commit Message** feature to summarize your final documentation updates.
- [ ] Send your completed project to GitHub:
  - [ ] `git add .`
  - [ ] `git commit -m "docs: finalized README and reflection"`
  - [ ] `git push origin main`

📍 **Checkpoint**: You've completed your first AI-assisted debugging project! Your game runs clearly, your documentation tells the story of your reasoning, and your reflection shows how you grew as an AI-aware engineer.

Close Section

▸ Optional Extensions

📬 Submitting Your Project

Once you've completed all the required features for your project, use the following checklist to prepare your work for submission.

1. Code is pushed to the correct GitHub repository
2. Repo is public
3. Required files are present (code, README, reflection, tests if applicable)
4. Commit history shows multiple meaningful commits
5. Reflection answers the prompts with specific, honest details
6. Final changes are committed and pushed before the deadline
7. Submit your assignment using the submit button.